

1. Extensibel: Module (Modularchitektur)
Die Hardware ist komplex. OS abstrahiert diese Komplexität. Betriebssysteme, erweiterte Schnittstelle zur Nutzung von Ressourcen.

2. Resource Manager
Kontrolliert und verwaltet Hardwarezugriff. Versteht die Komplexität, abstrahiert Zugriffsbedarf und priorisiert Nutzung.

3. Multi-User-System
User-Mode vs. Kernel-Mode: Schutzmechanismus. Northbound (User-Interface) vs. Southbound (HW-Interface).

CPU-Architektur & Emulation
CPU ist zentrale Rechenzentrale. Führt Programme abstrahiert aus. Führt zu Decoupling → Emulation.

4. Betriebssysteme
- Betriebssysteme (z.B. Windows, Linux, macOS)
- Betriebssysteme (z.B. Windows, Linux, macOS)
- Betriebssysteme (z.B. Windows, Linux, macOS)

Speicher-Management
- Hauptspeicher (RAM): Speicher für laufende Prozesse
- Sekundärer Speicher (SSD/HDD): Langfristige Speicherung
- Speicher-Management: Verwaltung von Speicherressourcen

System Calls
Prozesse dürfen nicht direkt auf Hardware zugreifen. OS stellt abstrakte Schnittstellen (System Calls) bereit.

Interpretation OS vs. Distribution
- Kernel: zentralisiertes Steuerungsprogramm (z.B. Linux-Kernel)
- Distribution: Kernel + Nutzer-Tools (z.B. Ubuntu)

Programme vs. Prozesse
- Programme: statische Dateien im Speicher
- Prozesse: laufende Instanzen von Programmen, aktiv vom Scheduler verwaltet

System Calls
- `systemctl` (manage services/units)
- `systemctl start/stop/restart` <service>
- `systemctl enable/disable` <service>
- `systemctl daemon-reload` (after create/edit)

Intros
- `journalctl` (view logs)
- `journalctl -u` <unit name> (full logs)
- `journalctl -p` <priority> (filter logs)

Grundlagen des Boot-Vorgangs
Beim Booten wird das Betriebssystem vom Grund auf gestartet. Der Prozess ist in Phasen gegliedert und dient dazu, System in betriebsfähigen Zustand zu versetzen.

1. Schritt - BIOS & POST (Basic IO)
ROM auf dem Mainboard gespeichert, führt ersten Code nach Einschalten aus. Führt POST durch: überprüft Hardware. (POST = Power-On-Self-Test). Erkennt angeschlossene Geräte über PCI-Bus-Scanning. Wählt Boot-Device über CMOS-Eintrag. Lädt ersten Sektor der gewählten Disk in RAM.

2. Schritt - Bootloader
MBR enthält den ersten kleinen Code, der Bootet lädt. (MBR = Master Boot Record). GRUB enthält den Speicherort des Kernels und lädt diesen ins RAM. Danach setzt Register und springt zur ersten Kernel-Instruction.

3. Schritt - Kernel-Initialisierung & System Aufbau
Sobald der Kernel im RAM liegt, startet er die Initialisierungsphase: Erkennt verfügbare Hardware und lädt passende Treiber. Initialisiert interne Datenstrukturen wie die Prozesstabelle. Startet Systemdienste. Startet ein User Interface.

Linux Boot-Prozess in 4 Schritten
1. BIOS/UEFI: Hardwareinitialisierung
2. GRUB: Auswahl und Laden des Kernels
3. Kernelstart: Initialisierung des Systems
4. mit Prozess: Startet Nutzerschnittstelle und Dienste

Boot
- 0. Start (Shut down)
- 1. Multi-user
- 2. Multi-user
- 3. Multi-user
- 4. User-Definable
- 5. Level 3 & GUI
- 6. Reboot

RAMDISK (Chicken-Egg-Problem)
OS braucht beim Start Zugriff auf Treiber/Module, die aber auf der Festplatte liegen. Lösung: initrd oder initramfs = temporäre Root-Dateisystem.

BIOS vs. UEFI

Problem BIOS	Lösung (UEFI)
16 Bit-Modus	64 Bit-Unterstützung
MBR mit 2TB-Grenze	GPT mit 2 ⁶⁴ Byte
Keine Erweiterbarkeit	Modular (Treiber, Secure Boot)
Beleider (Bootlits möglich)	Signierte EFI-Binaries

UEFI-Architektur und Praxis
- Führt eine architekturunabhängige VM aus, die EFI-Binary startet.
- EFI-Binaries liegen in der EFI System Partition mit FAT-Dateisystem.
- Mehrere OS können Einträge in die EFI-Liste eintragen. → ESP

EFI Boot-Manager
- verwaltet, welche EFI-Binary geladen wird.
- Jede OS-Installation registriert einen eigenen Eintrag.
- Einträge 0-9999 zeigt Bootreihenfolge und Einträge

Spezielle Nachteile
- Keine Dienst kennt den Zustand anderer Dienste → Race Conditions
- Keine einheitliche Abhängigkeitslogik → manuelle Skriptverwaltung
- Verteilungsfähige Startreihenfolge → schwer portierbar

System - Moderne Lösung
Nachfolger und übernimmt die gesamte Verwaltung des Systems. Parallelisierung des Systems (schnell). Verwaltung von Abhängigkeiten, Zuständen, Ressourcen. On-Demand-Abstraktion und geteiltes Banden von Diensten. Einheitliche Verwaltung aller Systeme über "Units". Units = zentrale Objekte in System.

Systemd - Unit-Zustände & Jobs
- active, inactive, activating/deactivating, failed
- Jobs: System verwaltet Statusänderungen als Job in einer Warteschlange. Jobs berücksichtigen Abhängigkeiten (z.B. After, Requires).

Service- & Target-Units
- S: Lege fest, wie ein Dienst gestartet, gestoppt, wird.
- T: Spezifizieren Abhängigkeiten.
- R: Gruppieren Units.
- T: Erstellen statischer Regelwerke.

Systemd - Abhängigkeiten
- Requires → Werte Abh. (muss vorhanden sein) → start parallel
- Wants → weiche Abh. (optional)
- Conflicts → negativ (wenn A läuft, darf B nicht laufen)
- Before, After → Reihenfolge (im Zeitlich, keine Abh.)
- Requires und Relativierung sind optional → müssen separat definiert werden.
- Viele Abhängigkeiten werden implizit erzeugt (z.B. Targets).

Was ist Scheduling?
Die zeitliche Zuweisung von Prozessen zu begrenzten Ressourcen - CPU.
- Ziel: optimale Ressourcennutzung
- Unterschiedliche Anforderungen je nach System

Begriffe aus der Queueing-Theorie
- Warteschlangenmodell: Prozesse warten in Queue auf Ressourcen.
- Scheduler trifft Auswahlentscheidungen basierend auf einer Policy.
- Dispatcher führt den Kontextwechsel durch.

Wann wird der Scheduler aktiv?
- Neue Prozesse, Prozess-Ende, Blockierung, Interrupts, Timer.
- Präemptiv Scheduler kann aktive Prozesse unterbrechen.
- Non-Preemptiv Prozess gilt Kontrolle freiwillig ab.

Scheduler-Strategien
- FIFO/FCFS: Nicht-präemptiv, sehr einfach.
- Prozesse laufen bis zur Beendigung.
- Problem: Lange Jobs blockieren kurze → schlechte Reaktionszeit.
- Round Robin: Präemptiv, Zeitscheiben-Verfahren mit Quantum q.
- Prozess läuft q ms; kein Verzug, fairer für interaktive Tasks.
- Multi-Level Queue: mehrere Queues für verschiedene Prioritäten.
- Round Robin: Präemptiv, Zeitscheiben-Verfahren mit Quantum q.
- Lösung: Multi-Level Feedback Queue.

Free Share Scheduling
- Alle Prozesse bekommen gleich viel CPU-Zeit → "Virtual Clock".
- Problem: bei hohem Lastgrad: schwer zu managen, hoher Overhead.

Hard vs. Soft Real-Time
- Hard RT: Deadline muss eingehalten werden.
- Soft RT: Deadline sollte eingehalten werden.
- Anforderung: deterministische Ausführung, kurze Latenz.

Rate Monotonic Scheduling (RMS)
- Feeste Priorität: kürzere Perioden → höhere Priorität.
- Generieren Plan mit zur Compile-Zeit festgelegten Tasks.
- nicht dynamisch zur Laufzeit.
- Garantierte Schließlichkeit nur bei $U \leq 0.5$ ($U = \sum \frac{C_i}{T_i}$).

Earliest Deadline First (EDF)
- Dynamischer Scheduler: Prozesse mit nächster Deadline bekommen CPU.
- Schedulbarkeit möglich bis 100% auslastung, aber schwer umzusetzen.

Policy Replacement
- RAM ist begrenzt → Welche Seite soll ersetzt werden?

Freeier Speicher & Swapping
- Freispeicherverwaltung muss effizient freie Bereiche finden.
- Bitmap: Kompakt, 1 Bit pro Block → schneller Zugriff.
- Versteckte Liste: Unterstützt First/Next/Best Fit.
- Swapping: temporäre Auslagerung von Prozessen auf Festplatte.
- Beim Neustart des Prozesses → Speicher wieder belegen.
- Ermöglicht parallele Ausführung vieler Prozesse.

Virtualer Speicher & Paging
- nutzt das Prinzip der lokalen Nutzung.
- Prozess wird in Pages aufgeteilt (nur benötigte Page geladen).
- Hauptspeicher wird in Frames aufgeteilt.
- Page Table verwaltet die Zuordnung.
- Resident Set: Alle aktuell im Speicher geladenen Seiten.
- Working Set: Aktuell verwendete Seiten.
- Page Replacement: RAM ist begrenzt → Welche Seite soll ersetzt werden?

Page Tables & MMU
- Problem: Page Tables liegen im RAM.
- Jeder Speicherzugriff braucht zwei Zugriffe:
1. Zugriff auf Page Table → Frame finden
2. Zugriff auf tatsächliche Daten
- Translation Lookaside Buffer (TLB):
1. TLB-Hit → schneller Zugriff
2. TLB-Miss → Zugriff auf Page Table
3. Page Table-Miss → Page Fault

Linux Memory Management
- Page Directory System: Hierarchisch aufgebaute Page Tables.
- Huge Pages: 2MB / 1GB Seiten (> weniger TLB-Einträge).
- Memory Zones: DMA, Normal, HighMem.
- Buddy Allocator: Blöcke mit 2ⁿ Seiten (z.B. für 5 → 2³).
- Slab Allocator: Für häufig genutzte kleine Objekte.
- Compaction: Speicher zusammenrücken.
- Shared Libraries: SL mit Position Independent Code.
- Reclaim: RAM zurückgewinnen durch Entfernen nicht aktiver Seiten.
- OOM Killer: Tötet Prozesse mit höchstem oom_score.

Working Set
- Working Set: Aktuell häufig genutzte Seiten.
- Resident Set: Alle Seiten im Speicher.

Prozesse & Prozessmodell
Ein Prozess ist ein laufendes Programm - eine aktive Einheit mit eigenem Zustand, Speicher und Kontext.
- Programm ≠ Prozess: Programm ist passiver Code; Prozess ist aktive Ausführung davon.
Prozess Aufbau, besitzt: Code, Daten, Stack, Heap.
Können sequentiell oder parallel ausgeführt werden.
Scheduler wählt aus, welcher Prozess laufen darf.
Jeder Prozess hat: Benutzerrechte (UID), Umgebungsvariablen, Modus (Vorfahr- & Hintergrund), Berechtigungen.

Prozess-Erzeugung / Prozesskilling
- Scheduler: Eintrag, Speicherabbild, Stack und Programmzähler gesetzt.
- Wechsel von System- in den User-Mode.
Typen der Prozessentscheidung:
- Systemstart, Benutzeranforderung, Erzeugung durch andere Prozesse, Capable Ausführung, Systeminterrupts.

Prozessbeendigungen
- Freiwillig: durch return aus main() oder exit().
- Freiwillig mit Fehler: z.B. Datei nicht gefunden.
- Unfreiwillig: durch Signale (z.B. kill -9).

Prozesszustände
- Running: CPU aktiv zugewiesen.
- Ready: Wartet auf Zuteilung der CPU.
- Blocked: Wartet auf Ereignis/Ressource.

Prozesszustandswechsel
- Auslöser für Zustandswechsel:
- Timer: Zeitquantum abgelaufen → zurück in 'Ready'.
- Interrupts: Ergebnisse von Hardware/Peripherie → oft 'Blocked' → 'Ready'.
- Page Fault: Daten nicht im RAM → Laden von Disk.
- System Calls: Freiwilliger Übergang in Kernel-Modus.

Kernel-Threads
Spezieller Thread, der ausschließlich im Kernel-Modus läuft.
- Hat PID, Zustand, PCB - wie normaler Prozess.
- Wird vom Scheduler verwaltet.
- Kein Wechsel in User-Mode möglich.

Kernel-Thread-Framework
- Workqueues: Aufgaben mit Priorität.
- Thread-Pool: Dynamisch verwaltet durch Daemon (kthread).

Speicherverwaltung
Elternprozess kann Kindprozess erzeugen → Erzeugung einer Prozesshierarchie.
Kind bekommt identisches Speicherabbild:
- Unkritisch: Read-Only
- Kritisch: Read-Write

User-Mode vs. System Mode
- U: Nur einfache Befehle
- S: Hardware zugriff erlaubt
- U: Kein Zugriff auf Hardware
- S: Hohe Privilegstufe
- S: Zugriff auf privilegierte Befehle

Mode Switch ≠ Context Switch
M: Wechselt zwischen User-/Kernel-Mode (z.B. bei Stackpoint).
C: Wechselt zwischen ausgeführten Prozessen/Threads, inkl. Register, Callpoint.

Thread-Modelle im Vergleich
User-Level-Threads (M:N)
- Verwaltung komplett im User Space.
- Das OS sieht nur einen einzigen Prozess → kein echtes Multithreading.
- Schnell & portabel.
- Kein Multicore-Support, ein blockierender Thread blockiert alle.
Kernel-Level Threads (1:1)
- Jeder Thread ist für das OS sichtbar und wird separat verwaltet.
- Unterschiedliche Multicore blockierende Threads führen anders nicht.
- Langsamer durch höheren Overhead & häufige Mode Switches.

Einführung in Groups
Control Groups ermöglichen Kontrollen über Ressourcen für Prozessgruppen.
- Organisieren Prozesse in Hierarchien.
- Kontrolle auf Prozess- & Thread-Ebene.

Groups v1 vs. v2
- v1: Jeder Controller kann eigene Hierarchie haben.
- v2: einheitliche Hierarchie für alle Controller.

Groups v1
- Ressourcenkontrolle individuell oder kollektiv.
- Groups = Verzeichnisse im cgroup-Dateisystem.
Jede Cgroup enthält Daten zur Konfiguration & Überwachung.
- v1 erlaubt unterschiedliche Groups für Threads eines Prozesses → problematisch.
- Erstellen "mkdir" & Befehl, Löschen "rmdir" Befehl; Befehl: `/sys/fs/cgroup/cpu/cg1`.

Mounting von Groups v1
- cgroups basieren auf treefs (temporäres Dateisystem).
- Ein Controller kann nicht mehrfach verwendet werden.
- mount -t cgroup -o cpu,cpuacct none /sys/fs/cgroup/cpu1,cpuacct → Einzelcontroller, (Container-Mount).

Unmounting
Cgroup-Filesystem kann mit "umount" entfernt werden, wenn:
- keine untergeordnete Groups mehr existiert.
- keine Prozesse (außer zombies) darin laufen.

Groups v1: Controller-Typen
- CPU Controller: Begrenzung von CPU-Zeit.
- CPUset Controller: Bindung von Prozess an bestimmte CPUs/NUMA-Knoten.
- Memory Controller: Begrenzung von RAM, Kernel-Memory und Swap-Nutzung.
- Blkio Controller: Begrenzung von Blockierzugriff.

Residets: CFS intern
- versucht Fairness herzustellen, indem er CPU-Zeit vergleicht.
- "Bigrat Losen" bekommt Vorrang.

Memory Management
Speicher ist eine Ressource des Systems, die vom OS verwaltet wird.
Memory Manager ist für den Hauptspeicher zuständig.

Fragmentierung
- Interne Fragmentierung: bei statischer Segmentierung bleibt ungenutzter Speicher im Segment.
- Externe Fragmentierung: bei dynamischer Zuteilung bleibt ungenutzter Speicher zwischen Blöcken.

Buddy-Algorithmus
- Rekursiver Algorithmus zur Speicherverwaltung.
- Zwei gleich große Blöcke (buddies) werden zusammengeführt.
- Speicher in 2ⁿ große Blöcke.
- Wenn 100k benötigt → 128k wird genommen.

Page Tables & MMU
- Problem: Page Tables liegen im RAM.
- Jeder Speicherzugriff braucht zwei Zugriffe:
1. Zugriff auf Page Table → Frame finden
2. Zugriff auf tatsächliche Daten
- Translation Lookaside Buffer (TLB):
1. TLB-Hit → schneller Zugriff
2. TLB-Miss → Zugriff auf Page Table
3. Page Table-Miss → Page Fault

Linux Memory Management
- Page Directory System: Hierarchisch aufgebaute Page Tables.
- Huge Pages: 2MB / 1GB Seiten (> weniger TLB-Einträge).
- Memory Zones: DMA, Normal, HighMem.
- Buddy Allocator: Blöcke mit 2ⁿ Seiten (z.B. für 5 → 2³).
- Slab Allocator: Für häufig genutzte kleine Objekte.
- Compaction: Speicher zusammenrücken.
- Shared Libraries: SL mit Position Independent Code.
- Reclaim: RAM zurückgewinnen durch Entfernen nicht aktiver Seiten.
- OOM Killer: Tötet Prozesse mit höchstem oom_score.

Working Set
- Working Set: Aktuell häufig genutzte Seiten.
- Resident Set: Alle Seiten im Speicher.

Prozessen zu Threads
Prozesse können mehrere Threads enthalten - Threads:
- Gehören zu einem Prozess → existieren nur innerhalb dessen.
- Haben sich Adressraum und Ressourcen.
- Haben eigenen Stack, Register, Programm Counter.
- Sehr leichtgewichtige.
- Ermöglicht parallele Verarbeitung innerhalb desselben Programms.
Vorteil: Nicht blockierend - auch wenn ein Thread auf I/O wartet, können andere weitermachen.

Resource Control
ps (process states)
s (interruptible sleep)
R (running)
D (uninterruptible sleep)
T (stopped)
Z (zombie, Defunct)

Nice-Wert & Einschränkung
Unix-Systeme erlauben es Benutzern, ihre Prozesse freiwillig zu "verlangsamen" (Nice-Wert erhöhen).
- "Nice"-Benutzer geben freiwillig CPU-Zeit ab.
- "Nur relative Werte, keine globale Fairness".
- Keine harte Durchsetzung, nur eigene Prozesse beeinflussen.

Kernel-Threads
Spezieller Thread, der ausschließlich im Kernel-Modus läuft.
- Hat PID, Zustand, PCB - wie normaler Prozess.
- Wird vom Scheduler verwaltet.
- Kein Wechsel in User-Mode möglich.

Kernel-Thread-Framework
- Workqueues: Aufgaben mit Priorität.
- Thread-Pool: Dynamisch verwaltet durch Daemon (kthread).

Thread-Modelle im Vergleich
User-Level-Threads (M:N)
- Verwaltung komplett im User Space.
- Das OS sieht nur einen einzigen Prozess → kein echtes Multithreading.
- Schnell & portabel.
- Kein Multicore-Support, ein blockierender Thread blockiert alle.
Kernel-Level Threads (1:1)
- Jeder Thread ist für das OS sichtbar und wird separat verwaltet.
- Unterschiedliche Multicore blockierende Threads führen anders nicht.
- Langsamer durch höheren Overhead & häufige Mode Switches.

Thread-Modelle im Vergleich
User-Level-Threads (M:N)
- Verwaltung komplett im User Space.
- Das OS sieht nur einen einzigen Prozess → kein echtes Multithreading.
- Schnell & portabel.
- Kein Multicore-Support, ein blockierender Thread blockiert alle.
Kernel-Level Threads (1:1)
- Jeder Thread ist für das OS sichtbar und wird separat verwaltet.
- Unterschiedliche Multicore blockierende Threads führen anders nicht.
- Langsamer durch höheren Overhead & häufige Mode Switches.

Thread-Modelle im Vergleich
User-Level-Threads (M:N)
- Verwaltung komplett im User Space.
- Das OS sieht nur einen einzigen Prozess → kein echtes Multithreading.
- Schnell & portabel.
- Kein Multicore-Support, ein blockierender Thread blockiert alle.
Kernel-Level Threads (1:1)
- Jeder Thread ist für das OS sichtbar und wird separat verwaltet.
- Unterschiedliche Multicore blockierende Threads führen anders nicht.
- Langsamer durch höheren Overhead & häufige Mode Switches.

Thread-Modelle im Vergleich
User-Level-Threads (M:N)
- Verwaltung komplett im User Space.
- Das OS sieht nur einen einzigen Prozess → kein echtes Multithreading.
- Schnell & portabel.
- Kein Multicore-Support, ein blockierender Thread blockiert alle.
Kernel-Level Threads (1:1)
- Jeder Thread ist für das OS sichtbar und wird separat verwaltet.
- Unterschiedliche Multicore blockierende Threads führen anders nicht.
- Langsamer durch höheren Overhead & häufige Mode Switches.

Thread-Modelle im Vergleich
User-Level-Threads (M:N)
- Verwaltung komplett im User Space.
- Das OS sieht nur einen einzigen Prozess → kein echtes Multithreading.
- Schnell & portabel.
- Kein Multicore-Support, ein blockierender Thread blockiert alle.
Kernel-Level Threads (1:1)
- Jeder Thread ist für das OS sichtbar und wird separat verwaltet.
- Unterschiedliche Multicore blockierende Threads führen anders nicht.
- Langsamer durch höheren Overhead & häufige Mode Switches.

Thread-Modelle im Vergleich
User-Level-Threads (M:N)
- Verwaltung komplett im User Space.
- Das OS sieht nur einen einzigen Prozess → kein echtes Multithreading.
- Schnell & portabel.
- Kein Multicore-Support, ein blockierender Thread blockiert alle.
Kernel-Level Threads (1:1)
- Jeder Thread ist für das OS sichtbar und wird separat verwaltet.
- Unterschiedliche Multicore blockierende Threads führen anders nicht.
- Langsamer durch höheren Overhead & häufige Mode Switches.

Ziel: Einheitliche Schnittstelle für Anwendungen

- Block Devices: Zugriff auf Blöcke fester Größe (Festplatte)
- Character Devices: Datenstrom Zeichen für Zeichen (Tastatur)

HW-Architektur

- I/O-Controller: Verwaltet Register und Puffer für Geräte
- I/O-Ports: Speicheradressen, die auf Controller-Register zeigen
- Assembler: OUT PORT, REG, MOV - Befehl

Speicherzuordnung

- Port-Mapped I/O: I/O-Register in eigenem Adressraum (nicht RAM)
- Memory-Mapped I/O: I/O-Register direkt in den Hauptspeicher gemappt

Direct Memory Access

- Gerät schreibt/liest direkt aus dem RAM - ohne CPU Hilfe
- CPU stellt Startadresse & Länge bereit → dann übernimmt Controller

Subsysteme & Device Model

- Ziel: Einheitliches, dynamisches Modell für alle Geräte
- device: physische Gerät
- driver: Software, die mit Gerät interagiert
- bus: Verbindungstyp
- class: Gerätegruppen mit ähnlichem Verhalten
- subsystem: logische Sicht auf das System

Eigener Linux Kernel

- Überflüssige Funktionen entfernen → kleinerer Kernel
- Unterstützung für spezielle Hardware aktivieren
- Eigene Funktionen einbauen
- Download über: wget https://...
- Entpacken mit: tar
- Menügeführte Konfiguration: make menuconfig

Zugriffsarten

- Logical vs Physical I/O: Logisches I/O abstrahiert von physischen Gerät
- Exclusive vs Shared Access: Nur ein Prozess oder mehrere gleichzeitig
- Sequential vs Random Access: FIFO-Datenstrom vs. beliebiger Zugriff
- Blocking vs Non-Blocking Access: Warten auf Antwort vs. sofortige Rückgabe
- Buffered vs Direct I/O: Zwischenspeicherung im RAM oder direkter Zugriff auf Gerät
- Error Handling: Fehler auf verschiedenen Ebenen → Soft- oder Hardwarebehandlung

Interrupt

1. Gerät sendet Signal (IRQ)
2. Interrupt Controller (PIC) wandelt IRQ in Vektor um
3. CPU liest Vektor, springt zur passenden Interrupt-Service-Routine
4. Nach Bearbeitung → Rückkehr zur vorherigen Ausführung

- Synchroner Interrupts: durch Programmveränderungen
- Asynchroner Interrupts: durch externe HW-Ereignisse
- Maskable Interrupts: können ignoriert werden
- Non-Maskable Interrupts: dürfen nicht ignoriert werden

Anforderungen an ein OS für Netzwerke

- Unterstützung von lokaler und verteilter Internetprozesskommunikation
- Integration von Netzwerkschnittstellen (NICs)
- Routing zwischen mehreren Interfaces
- Sicherheit: Schutz vor Netzwerkangriffen
- Virtualisierung: Netzwerknamespaces, virtuelle Interfaces für Cloud-Anwendungen

Layer 1- Physikalische Schicht & Netzwerkhardware

- Network Interface Card: Netzwerkkarte verbindet sich über den System-Bus mit System
- PHY (Physical Layer Device) - Schicht 1
- MAC (Media Access Controller) - Schicht 2

Physical Layer Device: verantwortlich für die physikalische Übertragung

Signalpegel, Clocking, Kodierung/Decodierung, Scrambling, Serialisierung

Stellt Bitstrom zur Verfügung → Übergabe an MAC

Media Access Controller: verarbeitet Ethernet-Frames

Empfängt Nibble-/Byte-Streams von PHY

Führt Integritätsprüfungen durch

Speichert Daten in Hauptspeicher - Buffer

Interrupt an NIC-Driver bei eingehenden Frames

Bei ausgehenden Frames: FCS hinzufügen, an PHY weiterleiten, Puffer überwachen

Layer 4- Transport-Schicht

UDP: Verbindungslos, einfaches Datagramm-Protokoll

Sanity-Checks, aber keine Garantie für Zustellung oder Reihenfolge

TCP: Verbindungsorientiert, zuverlässig

Verbindungsanbahn, Sequenzierung, Datenstrom-Wiederherstellung

Daten laden im TCP-Rx-Puffer → APP greift über Socket-API zu

Layer 7- Socket API

Berkeley Sockets - Schnittstelle für User-Programme

Verbindung zw. IP-Adresse, Port, Protokoll, Prozess

Socket-Typen: SOCK_STREAM(TCP), SOCK_DGRAM(UDP), SOCK_RAW(Ping)

Layer 2 - Datenlinkschicht, Zero-Copy & Bridging

- NIC-Treiber: Kernelmodul zur Steuerung der NIC
- Empfangseite (Rx): Prüft und korrigiert eingehende Frames
- Sendeseite (Tx): Initialisiert MAC für den Frameversand
- Netz Host-seitiges Buffering-System

Zero-Copy Stack: Effiziente Datenverarbeitung durch Minimierung von Kopiervorgängen

- Naive Architektur: Kopie des Frames bei jedem Layerwechsel
- Zero-Copy: Frame-Daten bleiben in Layer-2-Puffer, nur Pointer werden weitergegeben

sk_buff (SKB) - Buffering: Kernel-Datenobjekt für Network-Frames

- Enthält Metadaten, Payload-zeiger, Status
- Wird beim Empfang in Ringpuffer abgelegt (pro NIC)
- Über Scatter-Gather DMA (SG-DMA) direkt in SKB kopiert
- copy-from-user() überträgt am Ende Daten in User-Space

Bridging & Switching: Netzwerkschicht 2

- Bridging: Weiterleitung von Frames zwischen zwei Interfaces
- Bonding: NICs zusammenfassung für Load-Balancing
- Netfilter-Integration: für Filterregeln beim Bridging

Layer 3 - Netzwerkschicht & Routing

- Globale Adressierung über IP
- Routing über Subnetz-Kennung
- Firewall-Funktionalität durch Filterung basierend auf Paket-Metadaten
- Überprüfung und Weiterleitung von Frames → Layer 4

Netfilter: zentrales Framework für Filterung, NAT, Verbindungstracking

- Paketdurchlaufpunkte (Hooks): RRE-Routing, FORWARD, POST-Routing
- Das ip-rw() mit Netfilter auf → entscheidet über Annahme, Drop etc.

Return-Werte von Netfilter-Hooks:

- NF_DROP → Paket verwerfen, NF_ACCEPT → Verarbeitung fortsetzen
- NF_STOLEN → verarbeitet durch Hook, NF_QUEUE → Weiterleitung an Userspace
- NF_REPEAT → Funktion erneut ausführen

Datensysteme

Zentrale Bestandteile von Betriebssystemen zur Verwaltung persistenter Daten

- Strukturieren, speichern, schützen, effizient lesen/schreiben
- Mehrere Speichermedien unterstützen
- Zugriff über File System API → über OS abstrahiert

- Logische Sicht: Eine Datei ist eine benannte, abstrakte Informationseinheit
- Physische Schicht: Datei besteht aus Byte-Sequenz oder festen Records
- Dateitypen: Ordinary Files, Directory Files, Special Files

File Access

Sequenzieller Access: Daten werden in Reihenfolgen gelesen/geschrieben

Direkter Access: Zufälliger Zugriff auf bestimmte Positionen innerhalb der Datei

Metadaten

Protection	Owner/Group	Last mod. Date	LatFile
<code>rwX-rw-r--</code>	<code>1/Alma/Alma</code>	<code>53A/April 1 00:04</code>	<code>LatFile</code>
Links	Size in Bytes		Name

Implementierung von Verzeichnissen

- Dateiattribute speichern: Verzeichnisse müssen eine Abbildung von Dateinamen auf Dateiinformationen bereitstellen.
- Dateinamenslänge: Dateinamen sind in der Regel begrenzt, jedoch variieren die maximalen Längen je nach System.

Organisation auf Platten

- Physische Strukturierung: Dateien werden in Volumes, Partitionen und Verzeichnissen organisiert
- Flach: Alles auf einer Ebene (alte Kameras)
- Hierarchisch: Mit Root und Subdirectories
- Absolut: /home/user/datei.txt
- Relativ: ../tmp/datei.txt

ext-Dateisysteme

- ext 2: enthält Superblock, 1-Nodes, Datenblöcke, Blockgruppen
- Kein Journaling → bei Absturz riskant

ext 3: Erweiterung von ext 2 mit Journaling

ext 4: Nachfolger von ext 3

- Journaling über JBD, höhere Perf. & Zuverlässigkeit

Disk Space Management

- Zuweisung und Verwaltung von Speicher auf Blockbasis
- Kleine Blöcke → Weniger interne Fragmentierung, aber langsamerer Zugriff
- Große Blöcke → Höhere Datenrate, aber mehr Fragmentierung
- Verkettete Listen: Liste von freien Blöcken
- Bitmap: Abbildung freier und belegter Blöcke

Allokationsstrategien (Speichern von Dateien - S/D)

1. Contiguous Allocation: S/D im durchgehenden Blockbereich auf der Festplatte
2. Linked List Allocation: S/D als Kette einander Dateiblöcke
3. 1-Nodes: S/D mit Hilfe eines zentralen Indexknotens
4. keine externe Fragmentierung, einfache Verwaltung
5. Langsame zufällige Zugriffe, ineffiziente Nutzung des Speichers
6. keine externe Fragmentierung, schnelle Suche
7. Benötigt zusätzlichen Speicherplatz für 1-Nodes

Links

- Linking: Ein Verzeichnis kann auf eine Datei verweisen, sodass unter versch. Namen zugänglich
- Hard Links: Zeigen direkt auf denselben Inode
- Symbolische Links: Zeigen auf Pfad

Journaling

- Nach einem Systemabsturz darf das Dateisystem nicht inkonsistent sein
- Änderungen werden vorher in einem Log gespeichert
- Ausgeführt wird in atomaren Transaktionen

FHS-Filesystem Hierarchy Standard

- Festgelegte Struktur für Linux-Verzeichnisse (z.B. /bin, /usr, /var)

VFS-Virtual File-System

- Abstraktionsschicht: Einheitliche Schnittstelle trotz verschiedener Dateisysteme
- Solange ein konkretes System VFS-kompatibel ist, kann es eingebunden werden

Locking

- Mechanismus zur Vermeidung von Race Conditions bei parallelem Dateizugriff
- Details nicht ausgeführt, aber wichtig bei Multiuser-Umgebung

Nicht-persistente Dateisysteme (tmpfs)

- tmpfs speichert Daten im RAM
- Nicht persistent → Daten gehen beim Neustart verloren
- mount -t tmpfs -o size=2G tmpfs /mnt/tmp

LVM-Logical Volume Management

Zwischenschritt zwischen physischem Speicher und Dateisystem:

- PV (Physical Volume): eine echte Festplatte
- VG (Volume Group): Sammlung von PVs
- LV (Logical Volume): Virtuelle Partition innerhalb einer VG
- Vorteile: Dynamisches Resizing von Volumes, Balancing Snapshots über Copy-on-Write, Thin-Provisioning

Copy-on-Write

- Effizientes Kopieren von Daten
- Solange nur gelesen wird, wird keine Kopie erstellt
- Erst bei Änderungen wird die betroffene Datenstruktur dupliziert

Snapshots

- Momentaufnahmen des Zustands eines Volumes
- Kombination mit COW: Nur veränderte Blöcke werden kopiert
- Anwendung: Backup, Rollback

LVM-Funktionen

- Partitionen im laufenden Betrieb vergrössern/verkleinern
- Partitionen verschieben ohne Ausfallzeit
- Snapshots erzeugen, um Zustand einzufrieren

Btrfs (B-tree File System)

- COW-basiert, integriertes Volume Management
- Snapshots, RAID, Checksummen
- Online Resize, Kompression, Subvolumes
- Grundlage für Ceph

Ziele: Gute Performance auch bei Fragmentierung & Alterung

Für kleine bis grosse Systeme geeignet

Dateien & Verzeichnisse in Btrfs

- Werden in Filesystem Trees verwaltet
- Kleine Dateien können direkt im 1-Node gespeichert sein
- Grosse Dateien werden als Extents gespeichert

RAID

Kombination von Performance und Redundanz

- Durch Parity, Striping und Spiegelung
- RAID 0: Striping, keine Redundanz → schnell, aber riskant
- RAID 1: Spiegelung → hohe Ausfallsicherheit, halbiert Kapazität
- RAID 5: Striping + Paritätsdaten
- RAID 10: Kombination aus RAID 1 und 0

Btrfs Ablauf (Chunk)

- Btrfs teilt Geräte in grosse physische Chunks
- Diese werden zu logischen Chunks gruppiert
- Zuordnung: Chunk Tree: logisch → physisch; Device Tree: Physisch → logisch
- RAID wird auf Chunk-Level realisiert

RAID wird auf Chunk-Level realisiert *mv (moving file)*

Snapshots & Subvolumes

- Jeder Subvolume ist ein separater Baum im Btrfs-Wald
- Snapshots = konsistente Abbilder eines Subvolumes zu einem Zeitpunkt
- Schreibbare Snapshots = Clone

COW bei Update (Insert/Delete Keys)

- Änderungen führen nicht zu Überschreiben
- Stattdessen: neue Kopie → alte Blöcke bleiben erhalten
- Änderungen werden in Checkpoints gruppiert → konsistenter Zustand

Referenzierung (Cloning/Deleting)

- Neue Root zeigt auf gleiche Nodes wie Original
- Referenzzähler werden erhöht
- Beim Löschen: Zähler reduziert → bei 0 wird Speicher freigegeben

Defragmentation & Extent Relocation

Extents können verschoben werden, um Fragmentierung zu reduzieren

1. Extent K wird aus einem Chunk entfernt
2. Neue Kopie K' in anderem Chunk erstellt
3. Referenzbaum wird gelöst
4. Der neue Tree wird mit dem alten gemergt

Modifying Files

- Änderungen betreffen: Daten-Extents, Subvolume-Tree, Extent-Tree, Checksum-Tree
- Jeder dieser Bäume wird kopiert (COW) und neu aufgebaut

ls -i <files> (list inodes)

ln -s <target>file <link_name> (hard link)

ln -s <target>file <link_name> (soft link)

lsblk (all block devices)

sudo fdisk -l (partition tables)

sudo gdisk /dev/vdb (GPT partition table)

sudo mkfs.ext4 /dev/vdb1 (format ext4 filesystem)

sudo mount <partition> <mount-point>

sudo umount <mount-point>

sudo mount -t tmpfs -o size=2G tmpfs /mnt/mem (in-memory filesystem)

sudo mkfs.btrfs /dev/vdb1 (format btrfs)

sudo mount -o compress=zstd /dev/vdb1 /data (mount + compression)

sudo btrfs filesystem usage /data (btrfs usage)

sudo btrfs filesystem defragment -r -c <alg> /data (defragment)

ulimit -v <all> (limit memory)

-v (virtual memory)

-d (data segment)

-m (resident set RSS)

-s (stack size)

e.g. ulimit -v 50000

cat /proc/cgroups (supported controllers)

systemd-cgls (cgroup tree)

systemd-cgtop (live cgroup usage)

ps auxf -eo pid,user,cgroup,mgst (find cgroup)

sudo systemctl set-property user.slice CPUQuota=20% (limit user processes)

sudo systemctl run --slice=user.slice wc /dev/zero (run wc process in system)

Manual Cgroup Management

sudo mount -t cgroup -o cpuset none /mnt/my-cpuset (mount controller)

sudo mkdir /mnt/my-cpuset/group1 (create a cgroup)

echo 01 subo tee /mnt/my-cpuset/group1/cpuset.cpus (allowed cpu core)

echo 01 subo tee /mnt/my-cpuset/group1/cpuset.mems (allowed many nodes)

echo <PID> l subo tee /mnt/my-cpuset/group1/tasks (assign a process)

taskset -p <PID> (notify CPU affinity)

Overlays & Union-Dateisysteme

- Mehrere Verzeichnisse/Dateisysteme werden überlagert

- Ermöglicht: Schreiben auf schreibgeschützte Medien

- Container-Image: read-only (lowerdir)

- Container-Schreibbereich: read-write (upperdir)

mount -t overlay overlay -o lowerdir=/dir1:/dir2,upperdir=/upper,workdir=/work /merged